
Optimization via Greedy Oriented Progression

Xiaoli Li

Department of Statistics
The University of Chicago
Chicago, IL 60637, USA

Xiaohong Chen

Department of Economics
Yale University
New Haven, CT 06520, USA

Wei Biao Wu

Department of Statistics
The University of Chicago
Chicago, IL 60637, USA
wbwu@uchicago.edu

Abstract

Strategic Monte Carlo Optimization (SMCO) (Chen et al. [2026]) is a recent breakthrough for global optimization of high-dimensional multi-modal deterministic objectives via strategic averages. However, SMCO is not designed for online stochastic optimization and is computationally slow for huge-data empirical risk minimization problems. To resolve these drawbacks, we propose a novel *Greedy Oriented Progression* (GOP) algorithm, a first-order mini-batch optimization framework for high-dimensional multi-modal stochastic objectives. At each iteration, GOP converts the current stochastic descent direction into a boundary anchor by following the descent ray to the boundary of the feasible region, and then averages this anchor back to the interior. This boundary-seeking and recursive-averaging mechanism induces a natural “wide-to-narrow” trajectory, encouraging broad exploration in early iterations and stable local exploitation in later stages. Theoretically, under a Local Radial Alignment condition and stochastic directional-concentration assumptions, we show that GOP is locally stable, converges almost surely after entering a suitable basin, and achieves an $O(1/n)$ conditional mean-square rate. With a growing set of starting points, this yields an asymptotic global-minimizer guarantee. Empirically, we evaluate GOP and Adam-GOP on multi-modal synthetic functions, neural-network training tasks, and a large-scale real-data quantile-regression application. The results show that GOP variants achieve competitive or superior objective values with wall-clock costs comparable to standard stochastic first-order baselines such as SGD and Adam.

1 Introduction

Stochastic optimization lies at the heart of modern machine learning and large-scale statistical learning ([Bottou et al., 2018]). In many modern applications, one seeks to minimize an objective of the form

$$G(\theta) = \mathbb{E}[g(\theta, X)], \quad \theta \in \Gamma \subset \mathbb{R}^d,$$

where the feasible region Γ may be high-dimensional, the population objective G may be multi-modal and non-convex, and the data arrive sequentially or are too large to process at once. Designing reasonably fast algorithms that can reliably approach global solutions in such high-dimensional, multi-modal landscapes remains a well-known difficult problem; even non-convex quadratic programming is NP-hard in general ([Pardalos and Vavasis, 1991]). This difficulty is especially consequential in inference-oriented statistical problems, where numerical optimization error may directly propagate to parameter estimation and downstream inferential conclusions ([Tran et al., 2015]). For instance, likelihood-ratio statistics are nonnegative by construction when the likelihoods are globally maximized, but negative values can occur in practice when numerical optimization fails or returns a suboptimal local solution ([Shao, 2003, Douhard et al., 2017, Jacob et al., 2025]). This illustrates why local solutions produced by popular Stochastic Gradient Descent (SGD) based methods may be inadequate for inference-oriented objectives.

Our work is motivated by the recent significant breakthrough, Strategic Monte Carlo Optimization (SMCO) framework (Chen et al. [2026]), which provides a novel perspective on global optimization for multi-modal deterministic objectives. SMCO exploits strategic sampling and averaging principles to search beyond local basins. However, SMCO is designed for fixed objective functions and is not directly an online or mini-batch stochastic optimization algorithm. In particular, for empirical risk minimization with huge data sets, repeatedly evaluating or strategically sampling from the full objective can be computationally expensive. This raises a natural question: can the strategic-averaging intuition behind SMCO be converted into a scalable first-order stochastic online algorithm?

In this paper, we answer this question by proposing *Greedy Oriented Progression* (GOP), an online, mini-batch, gradient-based optimization framework. The key idea of GOP is simple. Let $\widehat{\nabla}G_n$ denote a stochastic gradient estimate at the current iterate θ_{n-1} . Instead of choosing only a scalar learning rate as in SGD, GOP first follows the stochastic descent direction until it reaches the boundary of the feasible region:

$$Z_n = \text{Boundary}(\theta_{n-1}, -\widehat{\nabla}G_n).$$

It then averages this boundary anchor back to the interior:

$$\theta_n = (1 - \alpha_n)\theta_{n-1} + \alpha_n Z_n, \quad \alpha_n = \frac{1}{n+1}.$$

Here $\text{Boundary}(\theta, v)$ denotes the boundary point obtained by moving from θ along direction v until the ray first reaches $\partial\Gamma$. Thus GOP turns each stochastic gradient direction into a boundary anchor and then recursively averages these anchors. This mechanism can be viewed as an online, gradient-driven analogue of the strategic averaging principle behind SMCO.

The resulting trajectory has a natural “wide-to-narrow” structure. In the early stages, the boundary anchors can be far from the current iterate, allowing the algorithm to explore a much wider region of the feasible set than a conventional local step. As the iteration index grows, the averaging weight shrinks automatically, so the algorithm gradually shifts from global exploration to local exploitation. In this sense, GOP is not merely SGD with a different learning-rate schedule: the step length is determined geometrically through a boundary-seeking operation, while the stabilization is provided by recursive averaging.

This boundary-seeking update also has two practical advantages. First, GOP is a low-cost first-order method with a modular direction oracle, so the raw stochastic gradient direction can be replaced by momentum or Adam-preconditioned directions, yielding variants such as Adam-GOP. Second, since GOP uses normalized directions, it is invariant to positive rescalings of the objective: replacing g by cg , $c > 0$, does not change the GOP trajectory. In contrast, such a rescaling changes the effective step size of SGD unless the learning rate is adjusted.

1.1 Contributions

The specific contributions of this work are four-fold:

- **Online SMCO-inspired optimization.** We introduce GOP, a first-order stochastic optimization algorithm that converts each mini-batch descent direction into a boundary anchor and then recursively averages these anchors. This provides an online and computationally scalable realization of the strategic-averaging intuition behind SMCO.
- **Boundary-seeking update with wide-to-narrow behavior.** Unlike SGD and its adaptive variants, GOP does not rely solely on a pre-specified scalar step size. Its update is determined by the geometry of the feasible region and the current stochastic descent direction. This creates a wide-to-narrow trajectory: early iterations encourage broad exploration, while later iterations automatically stabilize through averaging.
- **Theoretical guarantees under local geometric alignment.** We develop a theoretical foundation for GOP under a Local Radial Alignment condition and stochastic-gradient regularity assumptions. Our results show that mini-batch stochastic directions are uniformly aligned with population gradients, that GOP satisfies a local updating-alignment property, and that the resulting iterates converge almost surely with an $O(1/n)$ conditional mean-square rate after entering a suitable local basin. With a growing set of starting points, the theory further yields an asymptotic global-minimizer guarantee.

- **Empirical validation with real data analysis.** We validate GOP and Adam-GOP on multi-modal benchmark functions, high-dimensional neural-network training tasks, and a large-scale real-data quantile-regression application using IPUMS. The real-data experiment demonstrates that GOP variants remain competitive on a non-smooth, inference-oriented problem with millions of observations and wall-clock times comparable to SGD, Adam, and subgradient-based baselines.

1.2 Related Literature

Stochastic gradient methods. Stochastic Gradient Descent (Robbins and Monroe [1951]) and its variants remain the dominant optimization paradigm in modern machine learning. Momentum-based methods (Polyak [1964], Nesterov [1983]) accelerate optimization by accumulating past gradients, while adaptive methods such as AdaGrad (Duchi et al. [2011]), RMSProp (Tieleman and Hinton [2012]), Adam (Kingma and Ba [2015]), and AdamW (Loshchilov and Hutter [2019]) rescale coordinates based on historical gradient moments. Other robust variants, including sign-based methods (Bernstein et al. [2018], Moulay et al. [2019], Safaryan and Richtarik [2021]), have also been proposed to improve stability and communication efficiency. These algorithms are computationally efficient and easy to implement, but their trajectories remain primarily local and can be sensitive to step-size choices, initialization, and the geometry of non-convex landscapes. GOP uses the same first-order stochastic information as these methods, but replaces the usual scalar step-size decision by a boundary-seeking anchor followed by recursive averaging.

Global and non-convex optimization. A large literature studies algorithms for escaping saddle points and improving optimization in non-convex problems. Perturbed gradient descent (Jin et al. [2017]) and SGLD (Welling and Teh [2011]) use injected randomness to destabilize saddle points, while curvature-based methods such as cubic regularization (Nesterov and Polyak [2006]) exploit second-order information. These approaches are powerful but can require additional tuning or higher computational cost. Derivative-free global optimization methods provide broader exploration but often do not scale well to high-dimensional empirical risk minimization. In contrast, GOP remains a first-order method and is designed to be compatible with mini-batch and online computation.

SMCO and strategic averaging. The SMCO framework (Chen et al. [2026]) is closely related to the present work conceptually. It shows that strategic sampling and averaging can be used for global optimization of complicated deterministic objective functions. GOP builds on this principle but changes the computational regime. Rather than repeatedly operating on a fixed objective, GOP generates boundary anchors sequentially from stochastic gradient estimates and updates online. This makes the strategic-averaging idea applicable to huge-data empirical risk minimization and stochastic optimization problems. Moreover, compared with SMCO, GOP enjoys similar theoretical convergence guarantee under more natural and milder conditions, while still maintaining competitive performance on fixed objective functions.

2 Preliminaries

2.1 Problem Setting

To be specific, we consider the following general optimization problem

$$\theta^* \in \arg \min_{\theta \in \Gamma \subset \mathbb{R}^d} G(\theta), \text{ where } G(\theta) = \mathbb{E}_{X \sim \Pi}[g(\theta, X)]. \quad (1)$$

In (1), $\Gamma \subset \mathbb{R}^d$ is a compact convex set, $G : \Gamma \rightarrow \mathbb{R}$ is the differentiable objective function, and $g(\theta, X)$ is the noise-perturbed loss function with sequentially arriving i.i.d. random data $X_i \sim \Pi$ from some unknown distribution Π . We have access to $\nabla g(\theta, X) = \partial g(\theta, X)/\partial \theta = (\partial g(\theta, X)/\partial \theta_1, \dots, \partial g(\theta, X)/\partial \theta_d)^\top$, the stochastic gradient with respect to θ , as claimed by various SGD algorithms.

The compact feasible set Γ can often be introduced as a practical constraint or as an indicator-type regularization device. This setting covers many constrained empirical-risk minimization problems, while leaving unconstrained problems to be handled by choosing a sufficiently large computational domain.

2.2 Notation

For a vector $a = (a_1, \dots, a_d)^\top \in \mathbb{R}^d$ and $q > 0$, we denote $\|a\|_q = (\sum_{i=1}^d |a_i|^q)^{1/q}$, $\|a\|_\infty = \max_{1 \leq i \leq d} |a_i|$ and $\|a\| = \|a\|_2$. For any $s > 0$ and a random vector X , we say $X \in L_s$ if $\mathbb{E}[\|X\|_2^s] < \infty$. The symbols ∇ and ∇^2 represent the gradient vector and Hessian matrix of a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, respectively. For an event A , the indicator function that the event A happens is represented by $I\{A\}$. We use $\partial\Gamma$ to denote the boundary of any convex region Γ and $\mathcal{B}(a, r)$ to denote the common L_2 ball with center a and radius r .

3 Theory and Algorithms

3.1 Greedy Oriented Progression (GOP) Algorithm

We propose a novel gradient-based algorithm for optimizing target function with stochastic gradient, which is named Greedy Oriented Progression (GOP). Similarly to the stochastic gradient descent (SGD) method, it can be easily implemented under the parallel computing structure. The details of this method are shown in Algorithm 1.

Algorithm 1 Greedy Oriented Progression (GOP)

- 1: **Input:** Loss function $g(\theta; X)$, identically distributed independent data points $(X_1, \dots, X_N)$, batch size B , convex feasible region Γ , maximum number of iterations M and the initial point θ_0 .
 - 2: **for** $n = 1$ to $\min(M, \lfloor N/B \rfloor)$ **do**
 - 3: Calculate the noisy gradient vector direction $d_n = \frac{\sum_{i=(n-1)B+1}^{nB} \nabla g(\theta_{n-1}, X_i)}{\|\sum_{i=(n-1)B+1}^{nB} \nabla g(\theta_{n-1}, X_i)\|}$.
 - 4: Get the intersection point $Z_n \in \{\theta_{n-1} - td_n, t \geq 0\} \cap \partial\Gamma$.
 - 5: Update $\theta_n = \theta_{n-1} + \frac{1}{n+1}(Z_n - \theta_{n-1})$.
 - 6: **end for**
-

Compared to SGD, this method adaptively chooses the step size based on the current iteration number and the location. In other words, instead of moving along the gradient direction with a pre-specified step size, in each step we greedily progress to the boundary, which enables our algorithm to search widely in the initial few updates. Our GOP algorithm can also be interpreted as gradient optimal projection (GOP) or gradient oriented projection (GOP).

Due to the benefits of using the mini-batch trick, GOP can easily handle the large-scale optimization problems. In this efficient way that we input the stochastic gradient computed batch by batch, instead of the whole gradient, the computational cost is significantly reduced. Meanwhile, since we have many efficient algorithms, such as binary search, to compute the intersection point Z_n , our GOP algorithm can potentially be as fast as SGD.

3.2 Analysis on the Updating Mechanism

To effectively address the non-convexity of the problem, we introduce a set of general assumptions. Let $\mathcal{M} = \{\theta \in \Gamma : \|\nabla G(\theta)\| = 0\}$ represent the set of critical points, and $\mathcal{M}_0 \subset \mathcal{M}$ denote the set of local minima. To proceed, we impose the following standard regularity conditions on G and g , which are essential for establishing our results.

Assumption 3.1. The set $\mathcal{M}_0$ is non-empty and finite. Moreover, $\mathcal{M}_0 \cap \partial\Gamma = \emptyset$.

The assumption that $\mathcal{M}_0$ is non-empty is standard in the non-convex optimization literature, and appears in works such as Gitman et al. [2019], Jin et al. [2022] in the context of almost sure convergence of SGD. Additionally, we assume that $\mathcal{M}_0$ is finite. While this may seem restrictive, many well-known non-convex problems, such as matrix sensing, phase retrieval, matrix completion, and regression with non-convex constraints, satisfy this condition; see Ge et al. [2015], Tan and Vershynin [2023]. For neural networks, although saddle-point regions of positive measure may exist, it has been demonstrated that gradient-based algorithms like SGD can escape these regions with arbitrarily high probability Kleinberg et al. [2018], Wang et al. [2021]. The interior condition of $\mathcal{M}_0$ will be automatically satisfied if we push the boundary a little and make a smooth flat extension of

the function G Chen et al. [2026]. Therefore, Assumption 3.1 can be reasonably justified for most practical applications.

Furthermore, the optimization landscape of many non-convex statistical problems is often benign, allowing us to introduce additional assumptions regarding the behavior of G and g near the local minima. Specifically, we introduce the following condition.

Assumption 3.2. (Local Radial Alignment Condition). For each $\theta \in \mathcal{M}_0$, there exist $\delta(\theta) > 0$ and $c(\theta) > 0$ such that for all $\theta' \in \mathcal{B}(\theta, \delta(\theta))$,

$$\langle \theta' - \theta, \nabla G(\theta') \rangle \geq c \|\theta' - \theta\| \|\nabla G(\theta')\|. \quad (2)$$

Note that (2) is an acute angle condition. Geometrically, this assumption ensures that near the local minimum, the true gradient direction is strictly aligned with the position direction, i.e. the angle is strictly acute. Due to Assumption 3.1, we can assume $\delta(\theta) \equiv \delta$ and $c(\theta) \equiv c$ for all $\theta \in \mathcal{M}_0$.

Remark 3.3. Note that with local strong convexity and local Lipschitz continuity of $\nabla G(\theta)$ near $\mathcal{B}(\theta, \delta)$, we have

$$\nabla G(\theta')^\top (\theta' - \theta) \geq \mu \|\theta' - \theta\|^2 \geq \frac{\mu}{\mathcal{L}} \|\theta' - \theta\| \|\nabla G(\theta')\|, \quad (3)$$

where μ is the strong convexity constant and $\mathcal{L}$ is the Lipschitz constant. Thus, Assumption 3.2 holds for a broad class of optimization problems. Such localized assumptions are commonly employed in non-convex analysis, such as Yu et al. [2021], Zhong et al. [2023].

Next we analyze the iterates of GOP. Specifically, let $R(\theta, X) = \nabla G(\theta) - \nabla g(\theta, X)$ denote the gradient noise and

$$R_B(\theta) = B^{-1} \sum_{i=1}^B R(\theta, X_i), \quad (4)$$

which is the approximation error of the noisy gradient computed in a mini-batch with batch size B . To ensure the estimation error of GOP exhibits a quantifiable asymptotic behavior, it is crucial to control the randomized gradient $\nabla g(\theta_{n-1}, X_n)$. Thus we assume the exchangeability of integration and differentiation in the analysis, and impose mild smoothness conditions on the gradient noise $R(\theta, X)$.

Assumption 3.4. (Regularity of Gradient Noise). The function $g(\theta, X)$ is assumed to be continuously differentiable with respect to θ for any fixed X , and $E[\|\nabla g(\theta, X)\|^2]$ is also continuously differentiable. Additionally, it is assumed that $E[R(\theta_{n-1}, X_n) | \mathcal{F}_{n-1}] = 0$ for $\mathcal{F}_{n-1} = \sigma(\{\theta_0, \dots, \theta_{n-1}\})$.

Assumption 3.5. (Stochastic Local Hölder's Continuity). The gradient noise $R(\theta, X)$ satisfies

$$E[\sup_{\theta \in \Gamma} \|R(\theta, X_i)\|_q] < M, \quad q > 2, \quad (5)$$

for some $M > 0$. Moreover, there exists constants $\gamma' > 0$ and $\alpha' > 0$ such that for any fixed X ,

$$\|R(\theta, X) - R(\theta', X)\|_\infty \leq \mathcal{L}'(X) \|\theta - \theta'\|^{\alpha'}, \quad \text{for all } \theta, \theta' \in \Gamma \text{ with } \|\theta - \theta'\| \leq \gamma', \quad (6)$$

where $\mathcal{L}'(X)$ is a non-negative random variable with finite q -th moment.

Assumptions 3.4 and 3.5 consist of uniformly bounded q -th moment and local Hölder continuity of the gradient noise, which are widely used in the literature and can be found in works such as Zhu et al. [2023], Wei et al. [2025]. Building on these assumptions, we can develop a Fuk-Nagaev type inequality to bound the error $R_B(\theta)$, which is shown in Theorem 3.6.

Theorem 3.6. Suppose that the function g and G satisfy Assumptions 3.4 and 3.5. Then for any $y > 0$,

$$P \left[\sup_{\theta \in \Gamma} \|R_B(\theta)\|_\infty \geq c_1 d \log B / \sqrt{B} + y \right] \leq c_2 \frac{B^{1-q}}{y^q} + \exp(-c_3 y^2 B), \quad (7)$$

where c_1, c_2 and c_3 are positive constants independent of d and B . Consequently, for any $\varepsilon > 0$ and

$$\arcsin \left(\frac{2c_1 d^{3/2} \log B}{\varepsilon \sqrt{B}} \wedge 1 \right) \leq y \leq \pi/2,$$

we have

$$P \left[\sup_{\theta \in \Gamma} \omega(\theta) I\{\|\nabla G(\theta)\| > \varepsilon\} > y \right] \leq c_2 \frac{d^{q/2} B^{1-q}}{(\varepsilon \sin y)^q} + \exp(-c_3 B(\varepsilon \sin y)^2/d), \quad (8)$$

where

$$\omega(\theta) = \arccos \frac{\nabla G(\theta)^\top (\sum_{i=1}^B \nabla g(\theta, X_i))}{\|\nabla G(\theta)\| \|\sum_{i=1}^B \nabla g(\theta, X_i)\|}$$

is the angle between the true gradient $\nabla G(\theta)$ and the noisy gradient $\frac{1}{B} \sum_{i=1}^B \nabla g(\theta, X_i)$.

Theorem 3.6 implies that when the batch size B is large enough, the approximation error can be arbitrarily small and thus, the noisy gradient direction is well-aligned with the true gradient direction. It shows that the uniform error $\sup_{\theta \in \Gamma} \|R_B(\theta)\|_\infty$ is of probabilistic order $\mathcal{O}_P(d \log B / \sqrt{B})$. Consequently, with Assumption 3.2 on the true gradient, we can derive the following Lemma for the alignment of the updating direction, which is crucial for our theoretical analysis.

Lemma 3.7. *Suppose that the function g and G satisfy Assumptions 3.1, 3.2, 3.4 and 3.5. Then for any $\theta \in \mathcal{M}_0$ and $\varepsilon > 0$, there exist a batch size $B > 0$ and a radius $r > 0$, such that for any $n \in \mathbb{N}^+$, whenever $\theta_{n-1} \in \mathcal{B}(\theta, r)$ and $\|\nabla G(\theta_{n-1})\| \geq \varepsilon$, we have*

$$E[\langle \theta - \theta_{n-1}, \theta_n - \theta_{n-1} \rangle | \mathcal{F}_{n-1}] \geq \frac{c_4}{n+1} \|\theta_{n-1} - \theta\|, \quad (9)$$

where $c_4 > 0$ is a constant.

3.3 Theoretical Results on Convergence

As discussed in Theorem 3.6 and Lemma 3.7, under mild conditions, Eq. (9) holds, which implies that the updating direction $\theta_n - \theta_{n-1}$ will be well-aligned with the position vector $\theta - \theta_{n-1}$. Since Eq.(9) may also hold under other conditions, we state it as an intermediate assumption for generality and brevity.

Assumption 3.8. (Local Updating Alignment Condition) For each $\theta \in \mathcal{M}_0$, there exist $\delta'(\theta) > 0$ and $c'(\theta) > 0$ such that for any $n \in \mathbb{N}^+$, whenever $\theta_{n-1} \in \mathcal{B}(\theta, \delta'(\theta))$,

$$E[\langle \theta - \theta_{n-1}, \theta_n - \theta_{n-1} \rangle | \mathcal{F}_{n-1}] \geq \frac{c'}{n+1} \|\theta_{n-1} - \theta\|. \quad (10)$$

Next, based on the intermediate Assumption 3.8, we are ready to state our main theorems on the convergence of GOP.

Theorem 3.9. *Let G be a twice differentiable function on Γ and $L = \sup\{\|x - y\| : (x, y) \in \Gamma \otimes \Gamma\}$. Suppose that the functions g and G satisfy Assumptions 3.1 and 3.8. Then for any $\theta \in \mathcal{M}_0$, $\varepsilon > 0$ and $0 < r \leq \delta'(\theta)$, there exist $r_1 = r\sqrt{\varepsilon/2}$ and an integer $N_0 = \lceil \frac{2L^2}{r^2\varepsilon} \rceil$ such that for any $m \geq N_0$, whenever $\theta_m \in \mathcal{B}(\theta, r_1)$, the event $\Omega_{r,\theta} = \{\theta_n \in \mathcal{B}(\theta, r), \forall n \geq m\}$ occurs with probability at least $1 - \varepsilon$.*

Furthermore, given that $\Omega_{r,\theta}$ occurs, we have that

$$P(\lim_{n \rightarrow \infty} \theta_n = \theta \mid \Omega_{r,\theta}) = 1. \quad (11)$$

As a result, given a growing (deterministically, randomly, or quasi-randomly generated) set of starting points $\mathcal{X}_m$ such that $\#(\mathcal{X}_m) = m$ and $P(\mathcal{B}(\theta^*, r_1) \cap \mathcal{X}_m \neq \emptyset) \rightarrow 1$ as $m \rightarrow \infty$, where θ^* is a global minimizer, the GOP algorithm run from all points in $\mathcal{X}_m$ with a “boosted” starting iteration index $n_0 = N_0$ is guaranteed to attain a global minimizer θ^* asymptotically, as $n \rightarrow \infty$ and $m \rightarrow \infty$. The similar growing-start points strategy is frequently adopted in non-convex first-order optimization literature; see Chen et al. [2026].

Finally, we present a rate of convergence of our GOP algorithm given that $\Omega_{r,\theta}$ occurs, which is stated in the next Theorem 3.10.

Theorem 3.10. *Let G be a twice differentiable function on Γ and $L = \sup\{\|x - y\| : (x, y) \in \Gamma \times \Gamma\}$. Suppose that the functions g and G satisfy Assumptions 3.1 and 3.8 with the alignment parameter $c' > 0$. For any $\theta \in \mathcal{M}_0$, given that $\Omega_{\tilde{r},\theta}$ occurs, where $\tilde{r} \leq \min(c', \delta')$, then we have for all $n \geq m$,*

$$E[\|\theta_n - \theta\|^2] \leq \frac{n-m}{(n+1)^2} L^2 + \frac{(m+1)^2}{(n+1)^2} \tilde{r}^2. \quad (12)$$

4 Simulation Study

In this section, we present the results of the numerical experiments to empirically demonstrate the validity of our theory and empirical effectiveness of GOP. SMCO is not included as a main baseline because it is designed for fixed deterministic objectives and does not naturally operate in the online mini-batch regime considered here. Our empirical comparisons therefore focus on stochastic first-order methods with comparable access to mini-batch gradients. For brevity, the implementation details of algorithms, including discussion on the choice of hyperparameters such as the "learning rate" of GOP, and extra simulation results are deferred to Appendices B and D.

4.1 Random Optimization of Complicated Multimodal Functions

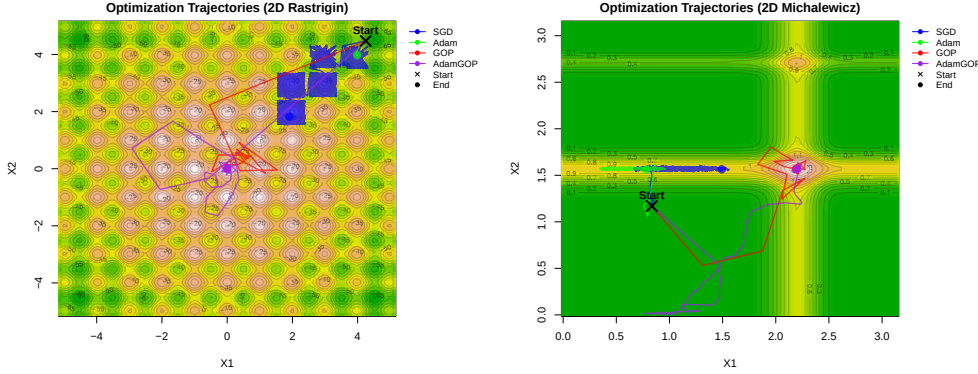


Figure 1: Trajectories of four optimization algorithms in a single run.

To investigate the global convergence properties of GOP in non-convex landscapes populated with numerous local optima, we benchmark its performance against standard stochastic optimizers and their variants. Specifically, we employ two classical non-convex benchmark functions known for their pathological landscapes: (1) The Rastrigin function on $[-5.12, 5.12]^d$. This function has a unique global minimum of value 0 at the origin, multiple global maxima, and many local optima. (2) The Michalewicz function on $[0, \pi]^d$, which features steep valleys and ridges, posing significant challenges for gradient-based methods to escape flat saddle points. This function has a unique global minimum, multiple global maxima, and many saddle points. To simulate the stochasticity inherent in mini-batch training, we inject isotropic Gaussian noise into the exact gradients; see Appendix B.

Figure 1 visualizes the optimization trajectories on 2D surfaces. A distinct contrast in behavior is observed: standard SGD and Adam rapidly converge to the nearest (suboptimal) local basin and stagnate. In contrast, GOP and Adam-GOP exhibit the "wide-to-narrow" search dynamic predicted by our theory.

Next, we assess the performance of these algorithms under various settings. The quantitative results for different dimensions and starting points are summarized in Table 1. In conclusion, our proposed algorithms show strong performance in these benchmarks when optimizing these complex functions, whereas baseline methods incur significant optimization error.

4.2 Experiments of Large Machine Learning Models

In this section, we assess the scalability and generalization capability of GOP on high-dimensional deep learning tasks.

We utilize two distinct neural network architectures trained on a synthetic regression task designed to mimic complex interactions: $y = \sin x_1 + \cos x_2 + x_3 x_4 + 0.1\mathcal{N}(0, 1)$, where $x \in \mathbb{R}^{10}$. The models include **DeepReLU MLP**: A 4-layer fully connected network with 64 hidden units, and **Transformer**: An encoder-only Transformer with 2 layers, 4 heads, and model dimension 32. We further scale this up to larger variants (up to 2×10^6 parameters) to test robustness.

Figure 2 illustrates the training dynamics in a single run. Across both architectures, GOP and Adam-GOP demonstrate superior convergence speeds, lower final Mean Squared Error (MSE) and

Table 1: Comparison with Four Stochastic Optimization Algorithms.

		Func.: Best Value	Rastrigin: 0			Michalewicz: -1.801		
		Algorithm	Regret	Dist.	Time	Regret	Dist.	Time
$d = 2$	Min	GOP	0.159	0.112	0.101	0.086	0.110	0.108
		ADAM-GOP	0.805	0.508	0.113	0.150	0.165	0.118
		SGD	31.613	2.206	0.090	0.653	0.716	0.094
		ADAM	17.58	2.757	0.096	0.665	0.730	0.098
	Max	Func.: Best Value	Rastrigin: 80.707			Michalewicz: 0		
		Algorithm	Regret	Dist.	Time	Regret	Dist.	Time
		GOP	2.894	–	0.101	0	–	0.091
		ADAM-GOP	1.909	–	0.108	7e-12	–	0.119
$d = 50$	Min	SGD	41.157	–	0.083	3e-4	–	0.094
		ADAM	23.859	–	0.087	4e-4	–	0.095
		Func.: Best Value	Rastrigin: 0			Michalewicz: -49.624		
		Algorithm	Regret	Dist.	Time	Regret	Dist.	Time
	Max	GOP	3.224	0.235	1.595	1.595	–	3.779
		ADAM-GOP	26.288	0.717	1.604	1.526	–	3.681
		SGD	788.78	2.370	1.530	3.714	–	3.529
		ADAM	440.13	2.934	1.516	1.645	–	3.579
$d = 100$	Min	Func.: Best Value	Rastrigin: 2017.665			Michalewicz: 0		
		Algorithm	Regret	Dist.	Time	Regret	Dist.	Time
		GOP	496.80	–	1.595	1e-12	–	3.612
		ADAM-GOP	490.11	–	1.604	8e-4	–	3.720
	Max	SGD	1031.58	–	1.530	0.031	–	3.665
		ADAM	595.26	–	1.516	0.014	–	3.670

Note: Based on 100 repetitions. In each repetition, all algorithms run with a single starting point drawn from Sobol’ sequence. For maximization experiments, the set of global maximizers may be non-unique or lie on the boundary; hence we report regret and time only. The exact global minimum of Michalewicz is unknown when $d = 50$.

similar computational cost compared to their baselines respectively. Specifically, in the Transformer task, Adam-GOP effectively combines the adaptive scaling of Adam with the global search of GOP, resulting in the most efficient optimization trajectory.

5 Real Data Application

5.1 Large-Scale Quantile Regression on IPUMS

We consider a quantile regression task based on the IPUMS USA data. IPUMS USA integrates individual-level U.S. census and American Community Survey microdata and is widely used in social, economic, demographic, and health-related research Ruggles et al. [2025]. For a fixed quantile level $\tau \in (0, 1)$, the empirical objective is the pinball-loss minimization problem

$$\min_{\theta \in \Gamma} \frac{1}{N} \sum_{i=1}^N \rho_{\tau}(Y_i - X_i^{\top} \theta), \quad \rho_{\tau}(u) = u\{\tau - I(u < 0)\}, \quad (13)$$

where Y_i is the response variable, $X_i \in \mathbb{R}^d$ is the covariate vector, and Γ is the optimization region used by GOP. Quantile regression is a classical tool for studying heterogeneous conditional effects and distributional features beyond the conditional mean Koenker [2005]. In large-scale applications, however, the non-smoothness of the pinball loss makes efficient and reliable optimization challenging. This benchmark is therefore well suited for evaluating GOP, since improved optimization quality can be consequential for downstream estimation and inference rather than merely predictive performance.

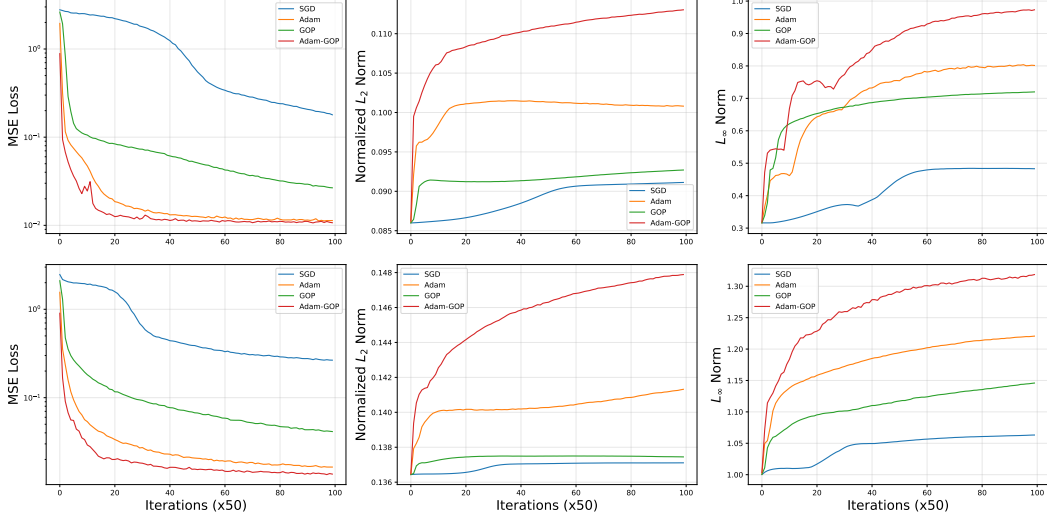


Figure 2: Performance comparison on DeepReLU MLP (first row) and Transformer (second row) in a single run. The first column shows the training loss; the second and third columns track the evolution of parameter norms. GOP variants consistently achieve lower loss.

The same IPUMS-type large-scale quantile regression task has been used in recent econometric work on fast inference for massive datasets, where subgradient descent (subGD) was proposed as a scalable baseline Lee et al. [2025]. Following this motivation, we compare GOP and Adam-GOP against SGD, Adam, and subGD. Additional implementation details are deferred to Appendix C.

5.2 Results and Practical Implications

Figure 3 reports the smoothed pinball-loss trajectories. The contrast between the baseline methods and GOP-type methods is clear. SGD decreases the objective slowly and remains substantially above the other methods for most of the optimization path. Adam improves substantially over SGD, but still exhibits a larger trajectory-averaged loss. In comparison, GOP, Adam-GOP, and subGD quickly enter the low-loss region, with GOP and Adam-GOP maintaining a strong advantage over standard SGD and Adam under comparable wall-clock time.

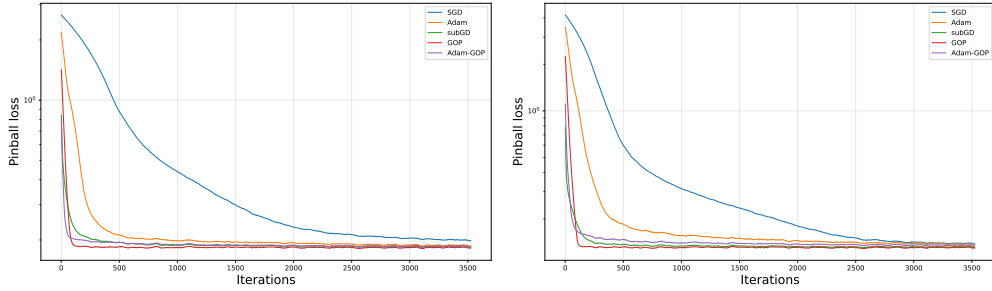


Figure 3: Real-data quantile regression on the IPUMS 1980 sample. The figure shows the smoothed pinball-loss trajectories of optimizers. Left: $\tau = 0.5$; right: $\tau = 0.8$. GOP-type methods rapidly enter the low-loss region and remain competitive throughout training.

Overall, this real-data experiment substantially strengthens the empirical evidence for GOP beyond the synthetic settings in Section 4. The IPUMS quantile-regression problem is large-scale, non-smooth, and directly connected to statistical estimation and inference. The results show that GOP-type updates can provide better optimization behavior than standard first-order baselines under nearly identical running time, while remaining competitive with the task-specific subGD baseline.

References

- Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Anima Anandkumar. signSGD: compressed optimisation for non-convex problems. In *International Conference on Machine Learning*, 2018. URL <https://api.semanticscholar.org/CorpusID:7763588>.
- Léon Bottou, Frank E Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018.
- Xiaohong Chen, Zengjing Chen, Wayne Yuan Gao, Xiaodong Yan, and Guodong Zhang. Optimization via the strategic law of large numbers. *Proceedings of the National Academy of Sciences*, 123(4):e2519845123, 2026. doi: 10.1073/pnas.2519845123. URL <https://www.pnas.org/doi/abs/10.1073/pnas.2519845123>.
- Victor Chernozhukov, Denis Chetverikov, and Kengo Kato. Inference on causal and structural parameters using many moment inequalities. *The Review of Economic Studies*, 86(5):1867–1900, 10 2019. ISSN 0034-6527. doi: 10.1093/restud/rdy065. URL <https://doi.org/10.1093/restud/rdy065>.
- Frédéric Douhard, Jean-Michel Gaillard, Maryline Pellerin, Laurent Jacob, and Jean-François Lemaître. The cost of growing large: Costs of post-weaning growth on body mass senescence in a wild mammal. *Oikos*, 126(9):1329–1338, 2017. doi: 10.1111/oik.04421.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.
- Rong Ge, Furong Huang, Chi Jin, and Yang Yuan. Escaping from saddle points - online stochastic gradient for tensor decomposition. In *Annual Conference Computational Learning Theory*, 2015. URL <https://api.semanticscholar.org/CorpusID:11513606>.
- Igor Gitman, Hunter Lang, Pengchuan Zhang, and Lin Xiao. Understanding the role of momentum in stochastic gradient methods. *Advances in Neural Information Processing Systems*, 32:9633–9643, 2019.
- Laurent Jacob, Frédéric Douhard, Jean-François Lemaître, Jean-Michel Gaillard, and Aurélie Siberchicot. *OnAge: Test of Between-Group Differences in the Onset of Senescence*, 2025. URL <https://CRAN.R-project.org/package=OnAge>. R package version 1.0.2.
- Chi Jin, Rong Ge, Praneeth Netrapalli, Sham M Kakade, and Michael I Jordan. How to escape saddle points efficiently. In *International Conference on Machine Learning (ICML)*, 2017.
- Ruinan Jin, Yu Xing, and Xingkang He. On the convergence of mSGD and AdaGrad for stochastic optimization, 2022. URL <https://arxiv.org/abs/2201.11204>.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Bobby Kleinberg, Yuanzhi Li, and Yang Yuan. An alternative view: When does SGD escape local minima? In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 2698–2707. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/kleinberg18a.html>.
- Roger Koenker. *Quantile Regression*. Number 38 in Econometric Society Monographs. Cambridge University Press, Cambridge, 2005. ISBN 9780521845731. doi: 10.1017/CBO9780511754098.
- Sokbae Lee, Yuan Liao, Myung Hwan Seo, and Youngki Shin. Fast inference for quantile regression with tens of millions of observations. *Journal of Econometrics*, 249:105673, 2025. doi: 10.1016/j.jeconom.2024.105673. Part A.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Emmanuel Moulay, Vincent Léchappé, and Franck Plestan. Properties of the sign gradient descent algorithms. *Information Sciences*, 492:29–39, 2019. ISSN 0020-0255. doi: <https://doi.org/10.1016/j.ins.2019.04.012>. URL <https://www.sciencedirect.com/science/article/pii/S0020025519303135>.
- Y. Nesterov. A method of solving the convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady*, 27(2):372–376, 1983.
- Yurii Nesterov and Boris T Polyak. Cubic regularization of newton method and its global performance. *Mathematical Programming*, 108(1):177–205, 2006.

- Panos M. Pardalos and Stephen A. Vavasis. Quadratic programming with one negative eigenvalue is NP-hard. *Journal of Global Optimization*, 1(1):15–22, 1991. doi: 10.1007/BF00120662.
- Boris T Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
- Herbert Robbins and Sutton Monro. A Stochastic Approximation Method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- Steven Ruggles, Sarah Flood, Matthew Sobek, Daniel Backman, Grace Cooper, Julia A. Rivera Drew, Stephanie Richards, Renae Rogers, Jonathan Schroeder, and Kari C. W. Williams. IPUMS USA: Version 16.0 [dataset], 2025. URL <https://doi.org/10.18128/D010.V16.0>.
- Mher Safaryan and Peter Richtarik. Stochastic sign descent methods: New algorithms and better theory. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9224–9234. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/safaryan21a.html>.
- Jun Shao. *Mathematical Statistics*. Springer, 2 edition, 2003.
- Yan Shuo Tan and Roman Vershynin. Online stochastic gradient descent with arbitrary initialization solves non-smooth, non-convex phase retrieval. *Journal of Machine Learning Research*, 24(58):1–47, 2023. URL <http://jmlr.org/papers/v24/20-902.html>.
- Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural networks for machine learning*, 4(2):26–31, 2012.
- Dustin Tran, Panos Toulis, and Edoardo M. Airolidi. Stochastic gradient descent methods for estimation with large data sets, 2015.
- Jun-Kun Wang, Chi-Heng Lin, and Jacob Abernethy. Escaping saddle points faster with stochastic momentum, 2021. URL <https://arxiv.org/abs/2106.02985>.
- Ziyang Wei, Wanrong Zhu, and Wei Biao Wu. Weighted averaged stochastic gradient descent: Asymptotic normality and optimality, 2025. URL <https://arxiv.org/abs/2307.06915>.
- Max Welling and Yee W Teh. Bayesian learning via stochastic gradient langevin dynamics. In *International Conference on Machine Learning (ICML)*, 2011.
- Lu Yu, Krishnakumar Balasubramanian, Stanislav Volgushev, and Murat A Erdogdu. An analysis of constant step size SGD in the non-convex regime: Asymptotic normality and bias. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 4234–4248. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/21ce689121e39821d07d04faab328370-Paper.pdf.
- Yanjie Zhong, Todd Kuffner, and Soumendra Lahiri. Online bootstrap inference with nonconvex stochastic gradient descent estimator, 2023. URL <https://arxiv.org/abs/2306.02205>.
- Wanrong Zhu, Xi Chen, and Wei Biao Wu. Online covariance matrix estimation in stochastic gradient descent. *Journal of the American Statistical Association*, 118(541):393–404, 2023.

A Theory and Proofs

A.1 Proof of Theorem 3.6

Proof. Let $\mathcal{N}_\epsilon$ be the smallest ϵ -net of Γ . Since Γ is compact, there exists some constant $L' > 0$ independent of d and ϵ , such that $|\mathcal{N}_\epsilon| \leq (\frac{L'}{\epsilon})^d$. Here we choose ϵ small enough such that $\epsilon \leq \gamma'$. Therefore, based on the Holder continuity of the gradient noise R , we have

$$\begin{aligned}
\sup_{\theta \in \Gamma} \|BR_B(\theta)\|_\infty &\leq \sup_{\theta \in \mathcal{N}_\epsilon} \|BR_B(\theta)\|_\infty + \sup_{\|u\| \leq \epsilon} \sup_{\theta \in \mathcal{N}_\epsilon} B\|R_B(\theta + u) - R_B(\theta)\|_\infty \\
&\leq \max_{1 \leq j \leq p} \left| \sum_{i=1}^B Y_{ij} \right| + \sum_{i=1}^B \mathcal{L}'(X_i) \epsilon^{\alpha'},
\end{aligned} \tag{14}$$

where $p = d|\mathcal{N}_\epsilon|$ and Y_i is the vectorized version of $(R(\theta, X_i))_{\theta \in \mathcal{N}_\epsilon}$. For the second term, since $\mathcal{L}'(X)$ is a non-negative integrable random variable, based on Markov's inequality and union bound, we have

$$P\left(\frac{1}{B} \sum_{i=1}^B \mathcal{L}'(X_i) > y\right) \leq BP(\mathcal{L}'(X_1) > y) \leq BE[|\mathcal{L}'(X_1)|^q]/y^q. \quad (15)$$

Next we deal with the first term. By using Lemma D.2 and Lemma D.3 in Chernozhukov et al. [2019], we have that for any $t > 0$,

$$\begin{aligned} P\left(\max_{1 \leq j \leq p} \left| \sum_{i=1}^B Y_{ij} \right| \geq 2K \left(\sigma \sqrt{\log p} + \sqrt{E[Q^2]} \log p \right) + Bt\right) \\ \leq e^{-(Bt)^2/(3\sigma^2)} + K_q BM/(Bt)^q, \end{aligned} \quad (16)$$

where K is a universal constant, K_q is a constant that solely depends on q , $\sigma^2 = \max_j \sum_{i=1}^B E[Y_{ij}^2]$ and $Q = \max_{1 \leq i \leq B} \max_{1 \leq j \leq p} |Y_{ij}|$. Because Assumption 3.4 ensures that $E[\sup_{\theta \in \Gamma} \|R(\theta, X_i)\|_q] < M$, we have $\sigma^2 \leq Bc_M$ and $E[Q^2] \leq BE[\max_j |Y_{ij}|^2] \leq Bc_M$. Here c_M is a constant that only depends on q and M . Therefore, we have

$$\begin{aligned} P\left[\sup_{\theta \in \Gamma} \|R_B(\theta)\|_\infty \geq 2K\sqrt{Bc_M} \left(\sqrt{\log p} + \log p \right) + t\right] \\ = P\left[\frac{1}{B} \left(\max_{1 \leq j \leq p} \left| \sum_{i=1}^B Y_{ij} \right| + \sum_{i=1}^B \mathcal{L}'(X_i) \epsilon^{\alpha'} \right) \geq 2K\sqrt{Bc_M} \left(\sqrt{\log p} + \log p \right) + t\right] \\ \leq P\left(\max_{1 \leq j \leq p} \left| \sum_{i=1}^B Y_{ij} \right| \geq 2K \left(\sigma \sqrt{\log p} + \sqrt{E[Q^2]} \log p \right) + Bt/2\right) + P\left(\frac{1}{B} \sum_{i=1}^B \mathcal{L}'(X_i) \epsilon^{\alpha'} \geq Bt/2\right) \\ \leq e^{-(Bt/2)^2/(3\sigma^2)} + 2K_q BM/(Bt/2)^q + BE[|\mathcal{L}'(X)|^q]/(B^2 t \epsilon^{-\alpha'}/2)^q \\ \leq \exp(-Bt^2/(12c_M)) + 2^{q+1}(K_q M + B^{-q} E[|\mathcal{L}'(X)|^q] \epsilon^{q\alpha'}) \frac{B^{1-q}}{t^q}. \end{aligned} \quad (17)$$

Note that by choosing $\epsilon = B^{-1/\alpha'}$, we have that $2K\sqrt{B^{-1}c_M} (\sqrt{\log p} + \log p) \leq 4K\sqrt{B^{-1}c_M} (\log d + d \log \mathcal{L}' + \frac{d}{\alpha'} \log B)$. Thus, by choosing $c_1 = 4K\sqrt{c_M}/\alpha'$, $c_2 = 2^{q+1}(K_q M + E[|\mathcal{L}'(X)|^q])$ and $c_3 = 1/(12c_M)$, we get the desired inequality.

Next, note that for any $\theta \in \Gamma$, $\arcsin\left(\frac{2c_1 d^{3/2} \log B}{\epsilon \sqrt{B}} \wedge 1\right) \leq y \leq \pi/2$ and $\epsilon > 0$, based on the geometry of the triangle

$$\begin{aligned} P\left(\sup_{\theta \in \Gamma} \arccos \frac{\nabla G(\theta)^\top (\sum_{i=1}^n \nabla g(\theta, X_i))}{\|\nabla G(\theta)\| \cdot \|\sum_{i=1}^n \nabla g(\theta, X_i)\|} \cdot I\{\|\nabla G(\theta)\| > \epsilon\} > y\right) \\ \leq P(\sup_{\theta \in \Gamma} \|R_B(\theta)\| > \epsilon \sin y) \\ \leq P(\sup_{\theta \in \Gamma} \|R_B(\theta)\|_\infty > \frac{\epsilon}{\sqrt{d}} \sin y) \\ \leq P(\sup_{\theta \in \Gamma} \|R_B(\theta)\|_\infty > c_1 d \log B / \sqrt{B} + \frac{\epsilon}{2\sqrt{d}} \sin y) \\ \leq c_2 \frac{d^{q/2} B^{1-q}}{(\epsilon \sin y)^q} + \exp(-c_3 \epsilon^2 \sin^2 y B/d). \end{aligned} \quad (18)$$

□

A.2 Proof of Lemma 3.7

Proof. Without loss of generality, we let $\theta = 0$. Consequently, we have for any $n \geq 1$,

$$\langle \theta - \theta_{n-1}, \theta_n - \theta_{n-1} \rangle = \langle -\theta_{n-1}, -\eta_n d_n \rangle = \eta_n \langle \theta_{n-1}, d_n \rangle, \quad (19)$$

where d_n is the noisy gradient unit and η_n is the adaptive step size at iteration t . Since Γ is compact, the step size $\eta_n = \|Z_n - \theta_{n-1}\|/(n+1) \leq L/(n+1)$, where L is the maximal diameter of Γ . Moreover, Assumption 3.1 ensures that θ is an interior point. Thus if $\theta_{n-1} \in \mathcal{B}(0, r)$, we also have

$$\eta_n = \|Z_n - \theta_{n-1}\|/(n+1) \geq l/(n+1), \quad (20)$$

where $l = \inf \text{dist}(\partial\Gamma, \mathcal{B}(0, \delta)) > 0$ by choosing r small enough. Next, based on Assumption 3.2, we have if $\theta_{n-1} \in \mathcal{B}(0, \delta)$,

$$\langle \theta_{n-1}, d_n^* \rangle \geq c\|\theta_{n-1}\|, \quad (21)$$

where d_n^* is the true gradient unit at iteration n . Then with Cauchy's inequality, we have

$$\begin{aligned} E[\eta_n \langle \theta_{n-1}, d_n \rangle | \mathcal{F}_{n-1}] &= E[\eta_n \theta_{n-1}^\top d_n^* | \mathcal{F}_{n-1}] + E[\eta_n \theta_{n-1}^\top (d_n - d_n^*) | \mathcal{F}_{n-1}] \\ &\geq \frac{cl}{n+1} \|\theta_{n-1}\| - \sqrt{E[\eta_n^2 | \mathcal{F}_{n-1}]} \sqrt{E(\|d_n - d_n^*\|^2 | \mathcal{F}_{n-1})} \|\theta_{n-1}\| \\ &\geq \frac{\tilde{c}}{n+1} \|\theta_{n-1}\|, \end{aligned} \quad (22)$$

where $\tilde{c} = cl - L \sqrt{\sup_{\theta \in \mathcal{B}(0, r), \|\nabla G(\theta)\| \geq \varepsilon} E[\|\frac{\nabla G(\theta)}{\|\nabla G(\theta)\|} - \frac{\sum_{i=1}^B \nabla g(\theta, X_i)}{\|\sum_{i=1}^B \nabla g(\theta, X_i)\|}\|^2]} > 0$ since according to Theorem 3.6, the second term is negligible when the batch size B is large enough and $\|\nabla G(\theta)\| \geq \varepsilon$. Specifically, note that

$$\begin{aligned} &\sup_{\theta \in \mathcal{B}(0, r), \|\nabla G(\theta)\| \geq \varepsilon} E \left[\left\| \frac{\nabla G(\theta)}{\|\nabla G(\theta)\|} - \frac{\sum_{i=1}^B \nabla g(\theta, X_i)}{\|\sum_{i=1}^B \nabla g(\theta, X_i)\|} \right\|^2 \right] \\ &\leq E \left[\sup_{\theta \in \Gamma} \left\| \frac{\nabla G(\theta)}{\|\nabla G(\theta)\|} - \frac{\sum_{i=1}^B \nabla g(\theta, X_i)}{\|\sum_{i=1}^B \nabla g(\theta, X_i)\|} \right\|^2 I\{\|\nabla G(\theta)\| \geq \varepsilon\} \right] \\ &\leq E \left[\sup_{\theta \in \Gamma} |\omega(\theta)| I\{\|\nabla G(\theta)\| \geq \varepsilon\}^2 \right]. \end{aligned} \quad (23)$$

Since the inner part is upper bounded by π^2 and $\lim_{B \rightarrow \infty} \sup_{\theta \in \Gamma} |\omega(\theta)| I\{\|\nabla G(\theta)\| \geq \varepsilon\} = 0$ according to (18), based on Lebesgue's dominant convergence theorem, we have

$$\lim_{B \rightarrow \infty} \sup_{\theta \in \mathcal{B}(0, r), \|\nabla G(\theta)\| \geq \varepsilon} E \left[\left\| \frac{\nabla G(\theta)}{\|\nabla G(\theta)\|} - \frac{\sum_{i=1}^B \nabla g(\theta, X_i)}{\|\sum_{i=1}^B \nabla g(\theta, X_i)\|} \right\|^2 \right] = 0. \quad (24)$$

□

A.3 Proof of Theorem 3.9

Proof. Without loss of generality, we let $\theta = 0$. Let $V_n = \|\theta_n\|^2$, the squared L_2 distance, as the Lyapunov function. Consequently, we have for any $n \geq 1$,

$$V_n = \|\theta_n\|^2 = V_{n-1} - 2\eta_n \langle \theta_{n-1}, d_n \rangle + \eta_n^2, \quad (25)$$

where d_n is the noisy gradient unit and η_n is the adaptive step size at iteration n . Since θ is an interior point and Γ is compact, the step size $\eta_n = \|Z_n - \theta_{n-1}\|/(n+1) \leq L/(n+1)$, where L is the maximal diameter of Γ .

Next, we define the stopping time $\tau = \inf\{n > m : \|\theta_n\| > r\}$ and let $Y_n = \|\theta_{n \wedge \tau}\|^2 + \sum_{k=n \wedge \tau}^\infty L^2/(k+2)^2$. Then

$$\begin{aligned} E[Y_{n+1} | \mathcal{F}_n] &= E[\|\theta_{(n+1) \wedge \tau}\|^2 | \mathcal{F}_n] + \sum_{k=(n+1) \wedge \tau}^\infty \frac{L^2}{(k+2)^2} \\ &\leq Y_n I\{n \geq \tau\} + \left(\|\theta_n\|^2 + E[\eta_{n+1}^2 | \mathcal{F}_n] + \sum_{k=n+1}^\infty \frac{L^2}{(k+2)^2} \right) I\{n < \tau\} \\ &\leq Y_n, \end{aligned} \quad (26)$$

where the first inequality holds since if $n < \tau$, $\theta \in \mathcal{B}(0, r)$ and thus

$$E[\eta_{n+1} \langle \theta_n, d_{n+1} \rangle | \mathcal{F}_n] \geq \frac{c'}{n+2} \|\theta_n\|, \quad (27)$$

as implied by Assumption 3.8. Therefore, Y_n is a non-negative supermartingale with respect to $\mathcal{F}_n$. Then by applying Doob's maximal inequality, we have

$$P(\tau < \infty) = P(\sup_{n > m} \|\theta_{n \wedge \tau}\| > r) \leq P(\sup_{n > m} Y_n > r^2) \leq \frac{E[Y_m]}{r^2} \leq \frac{r_1^2 + \sum_{k=N_0}^{\infty} \frac{L^2}{(k+2)^2}}{r^2} \leq \varepsilon, \quad (28)$$

when $r_1 = r\sqrt{\varepsilon/2}$ and $N_0 = \lceil \frac{2L^2}{r^2\varepsilon} \rceil$. Therefore, the event $\Omega_{r,0}$ occurs with probability at least $1 - \varepsilon$.

Next, given that the event $\Omega_{r,0}$ occurs, we have that Eq. (27) holds according to Assumption 3.8. Thus,

$$E[V_{n+1} | \mathcal{F}_n] \leq V_n - \frac{2c'}{n+2} \|\theta_n\| + \frac{L^2}{(n+2)^2}. \quad (29)$$

Since $\sum_n L^2/(n+2)^2 \leq \infty$, by the Robbins-Siegmund Theorem, we have that $V_n \rightarrow V_\infty$ and $\sum_n 2c' \|\theta_n\|/(n+2) < \infty$ almost surely, where V_∞ is a finite random variable. Note that $\sum_n 2c' \|\theta_n\|/(n+2) < \infty$ implies that $\liminf_{n \rightarrow \infty} \|\theta_n\| = 0$ since $c' > 0$. Thus, we must have $V_\infty = 0$ almost surely, which implies that $\lim_{n \rightarrow \infty} \theta_n = 0$, i.e. the almost surely convergence. $\square$

A.4 Proof of Theorem 3.10

Proof. Without loss of generality, we will take $\theta = 0$. Given $\Omega_{\tilde{r},\theta}$, define $V(\theta) = \|\theta\|^2$. Denote $S_k = \sum_{t=0}^k Z_t$, where $Z_0 = \theta_0$. Therefore we have $V(0) = 0$, $S_k = (k+1)\theta_k$. First we consider the unconditional event. Define the stopping time $\tau = \inf\{n > m : \|\theta_n\| > \tilde{r}\}$. By Taylor's formula,

$$\begin{aligned} E[V(\theta_{n \wedge (\tau-1)})] &= \sum_{k=m+1}^n E \left[V\left(\frac{S_{k \wedge (\tau-1)}}{n+1}\right) - V\left(\frac{S_{(k-1) \wedge (\tau-1)}}{n+1}\right) \right] + E \left[V\left(\frac{(m+1)\theta_{m \wedge (\tau-1)}}{n+1}\right) \right] \\ &= \sum_{m < k \leq n \wedge (\tau-1)} E \left[\frac{Z_k}{n+1} \cdot D_\theta V\left(\frac{S_{(k-1)}}{n+1}\right) \right] + \sum_{k=m+1}^n \frac{E[\|Z_{k \wedge (\tau-1)}\|^2]}{(n+1)^2} + \frac{(m+1)^2}{(n+1)^2} E[\|\theta_m\|^2] \\ &= \sum_{m < k \leq n \wedge (\tau-1)} \frac{2k}{(n+1)^2} E[\theta_{(k-1)} \cdot Z_k] + \sum_{k=m+1}^n \frac{E[\|Z_{k \wedge (\tau-1)}\|^2]}{(n+1)^2} + \frac{(m+1)^2}{(n+1)^2} E[\|\theta_m\|^2] \\ &\leq \frac{n-m}{(n+1)^2} L^2 + \frac{(m+1)^2}{(n+1)^2} \tilde{r}^2 = O\left(\frac{1}{n}\right). \end{aligned} \quad (30)$$

The last inequality holds because $\|Z_k\| \leq L$, $\|\theta_m\| \leq \tilde{r}$ and for all $m < k < \tau$

$$E[\theta_{k-1}^\top Z_k] = E[E[\theta_{k-1}^\top (\theta_{k-1} - (k+1)\eta_k d_k) | \mathcal{F}_{k-1}]] \leq E[\|\theta_{k-1}\|^2 - c' \|\theta_{k-1}\|] \leq 0, \quad (31)$$

due to Assumption 3.8 and that $\|\theta_{k-1}\| \leq \tilde{r} \leq c'$ when $k < \tau$. Consequently, given that $\Omega_{\tilde{r},\theta}$ occurs, we have $\tau = \infty$ and $V(\theta_{n \wedge (\tau-1)}) = V(\theta_n)$ almost surely, from which the conditional local linear convergence rate follows. $\square$

B Details of Simulation Study

B.1 Practical Implementation of the GOP Algorithm

In this subsection, we describe in more details how we practically implement Algorithm 1 in R and Python, and introduce its variant, namely ADAM-GOP, which we use for the numerical experiments in Section 4. Algorithm 2 details the practical implementation of GOP. ADAM-GOP has the same updating mechanism as GOP, except for the progression direction, which is the same as ADAM instead of the vanilla gradient.

Algorithm 2 Practical Greedy Oriented Progression

```
1: Input: Loss function  $g(\theta; X)$ , identically distributed independent data points  $(X_1, \dots, X_N)$ ,  
   batch size  $B$ , threshold parameter  $\varepsilon$ , convex feasible region  $\Gamma$ , maximum number of iterations  
    $M$ , initial point  $\theta_0$ , starting index  $m$  and buffer parameter  $\delta$ .  
2: for  $n = 1$  to  $K = \min(\lfloor cM \rfloor, \lfloor cN/B \rfloor)$  do  
3:   Calculate the stochastic gradient vector direction  $d_n =$   
    $\sum_{i=(n-1)B+1}^{nB} \nabla g(\theta_{n-1}, X_i) / \max(\|\sum_{i=(n-1)B+1}^{nB} \nabla g(\theta_{n-1}, X_i)\|, \varepsilon)$ .  
4:   Calculate the intersection time  $t_n = \inf\{t : \theta_{n-1} - td_n \in \partial\Gamma, t \geq 0\}$ .  
5:   Draw the perturbed intersection point  $Z_n = \theta_{n-1} - (1 + \xi_n)t_n d_n, t \geq 0$ , where  $P(|\xi_n| \leq$   
    $\delta) = 1$ .  
6:   Update  $\theta_n = \theta_{n-1} + \frac{1}{n+m}(Z_n - \theta_{n-1})$ .  
7:   if  $\theta_n \notin \Gamma$  then  
8:     Update  $\theta_n \in \{\theta_{n-1} + t(Z_n - \theta_{n-1}), t > 0\} \cap \partial\Gamma$ .  
9:   end if  
10: end for
```

Motivated by Theorem 3.9, we introduce the starting index m for a more local search. Specifically, given any starting point θ_0 , it initializes the running sum as $S_0 = m\theta_0$, computes the first update as $(S_0 + Z_1)/(m + 1)$, and so on and so forth. We also introduce extra randomness, represented by ξ_n to enlarge the original region so that the algorithm does not get stuck if the minimum is on the boundary. The computation of the intersection time t_n is simply a line search, whose computational cost is negligible as long as the region Γ is convex.

It is worth noting that the parameter m naturally enables us to fairly compare our GOP with baselines. For example, with a small m , the "step size" of GOP behaves similarly to a large fixed initial learning rate with polynomial decay, while a large m makes it almost constant. Moreover, with minor modifications, all of our theoretical results in Section 3.2 and Section 3.3 will also hold for this practical GOP, since our analysis is robust to the initial point and enlargement of the boundary.

Moreover, the GOP algorithm can be further improved if we use a refined version called GOP-R. First, GOP-R records and exploits the running maximum of all test function evaluations: This is purely bookkeeping and does not add any additional evaluations of the test functions beyond those for finite-difference calculations and convergence checks. From a programming perspective, the running maximum generated in past iterations is "free information" that requires essentially no additional computation and memory storage. Second, GOP-R adopts a two-stage procedure: it starts by running GOP for at most 50% iterations of the designated maximum number of iterations, and then runs the rest 50% iterations with a more local search (and smaller step sizes). Other hyperparameters are set to be the same as in GOP. The popular Cosine Annealing learning rate scheduler for SGD is a natural baseline for GOP-R. We leave the discussion and experiments about GOP-R for future work.

B.2 Discussion on Choice of Hyperparameters

We now describe the default choice of hyperparameters for our GOP algorithms. The batch size B is expected to be chosen reasonably large, as suggested in most modern stochastic optimization settings. For the random variables ξ_n and the buffer parameter δ introduced in Algorithm 2, we set $\xi_n \sim \text{Unif}(-\delta^*, \delta^*)$ with default $\delta^* = 0.05$, i.e., 5% of the length of the domain.

For the choice of region Γ , if the original problem is a convexly-constrained optimization problem, the region defined by the constraints is a natural choice. For general unconstrained optimization problems, hyper-cube centered at the initial point is a recommended option. In particular, if the user has prior information or belief about the upper bound of L_∞ norm of the target minimum, i.e. possible largest absolute value of the coordinates, then Γ can be set slightly larger than that domain.

For example, if the initial point is a warm start, a natural choice of Γ is a small hyper-cube around the warm start. This is a very common case in Reinforcement Learning, where the user is often asked to fine-tune a pre-trained model. A more practical but less efficient way is to first run the initial few steps of standard unconstrained optimizers like SGD and track the evolution of L_∞ norm in the early stage. In this case, Γ can be chosen as a hyper-cube which contains the trajectory. Moreover, our

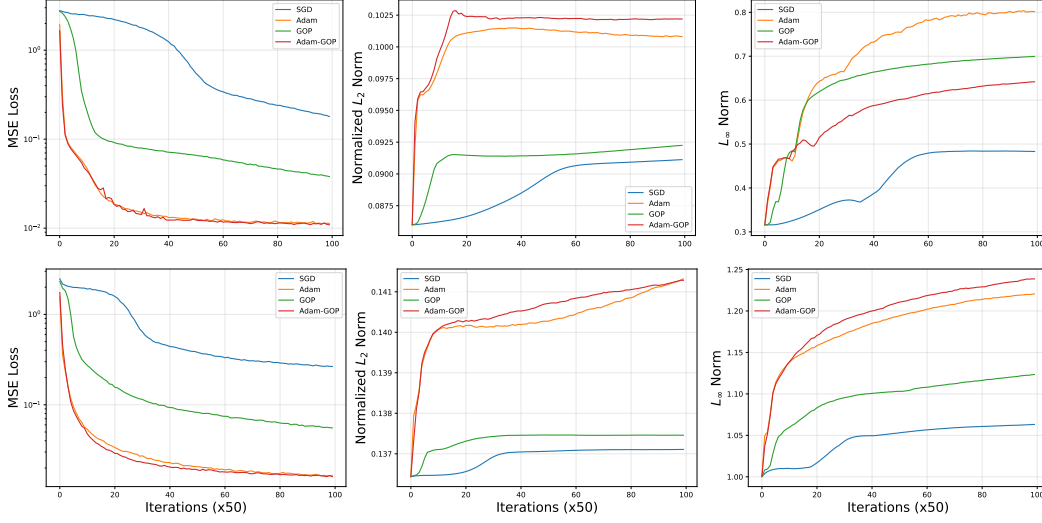


Figure 4: Performance comparison on DeepReLU MLP (first row) and Transformer (second row) in a single run. The first column shows the training loss; the second and third columns track the evolution of parameter norms. GOP variants consistently achieve lower loss. For DeepReLU MLP, the total dimension is $d \sim 1.3 \times 10^4$ and the training costs are (46, 49, 50, 52) seconds respectively. For Transformer, the total dimension is $d \sim 1.7 \times 10^4$ and the training costs are (103, 119, 117, 125) seconds respectively.

simulation results show that the performance of GOP is robust to the choice of the edge length of hyper-cube Γ .

B.3 Details on the Simulation in Section 4.1

As discussed in Section 4.1, in each step the stochastic optimization algorithms receive a corrupted gradient $\nabla g(\theta, \epsilon_n) = \nabla G(\theta) + \sigma \epsilon_n$, where $\epsilon_n \sim \mathcal{N}(0, I_d)$ represents the noise at step n and σ denotes the noise level. In particular, we choose $\sigma = 1$ to model the moderately strong noise case. We repeat 100 times with varying starting points drawn from Sobol' sequence. The random seed used is 123. For SGD and ADAM, they solve an equivalent Lagrangian problem $\min_{\theta} G(\theta) + \lambda^{\top} h(\theta)$, where the regularizer λ is chosen as 100 and $h(\theta)$ is the function of constraints. This penalty method does not unfairly disadvantage the baselines since the constraints are always inactive during the whole experiments.

We compare the following 4 algorithms.

- **SGD:** Stochastic Gradient Descent by Robbins and Monro [1951]. We implement this algorithm in R with learning rate 0.01 and max number of iterations 5,000.
- **ADAM:** Adaptive Moment Estimation method by Kingma and Ba [2015]. We implement this algorithm in R with learning rate 0.01, max number of iterations 5,000 and ADAM parameters $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$.
- **GOP:** Greedy Oriented Progression as in Algorithm 2. The tuning parameters are set as $m = 0$ and $\delta = 0.05$ with max number of iterations 5,000.
- **ADAM-GOP:** Greedy Oriented Progression with ADAM direction. The parameters are chosen the same as GOP with ADAM parameters $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$.

The regret is defined as the average of absolute differences between global minimum/maximum value and the output given by the algorithm. The distance is defined as the average of normalized Euclidean distance between the global minimizer/maximizer and the point found by the algorithm.

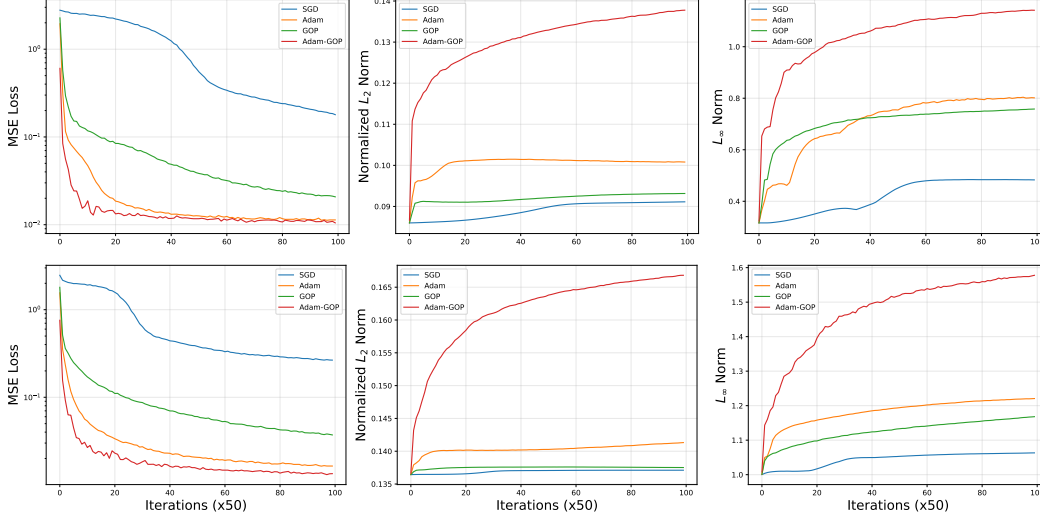


Figure 5: Performance comparison on DeepReLU MLP (first row) and Transformer (second row) in a single run. The first column shows the training loss; the second and third columns track the evolution of parameter norms. GOP variants consistently achieve lower loss. For DeepReLU MLP, the total dimension is $d \sim 1.3 \times 10^4$ and the training costs are (46, 49, 51, 55) seconds respectively. For Transformer, the total dimension is $d \sim 1.7 \times 10^4$ and the training costs are (109, 182, 178, 189) seconds respectively.

B.4 Details on the Simulation in Section 4.2

The synthetic data are generated as $y = \sin x_1 + \cos x_2 + x_3 x_4 + 0.1 \mathcal{N}(0, 1)$ with $x \sim 4\mathcal{N}(0, I_{10}) - 2$. The models include **DeepReLU MLP**: A n -layer fully connected network with d hidden units, and **Transformer**: An encoder-only Transformer with n layers, m heads, and model dimension d . We let $n \in \{2, 4, 8\}$, $m = 4$ and $d \in \{32, 64, 128, 256, 512\}$. Due to limit of computational resources, larger models are not considered.

All models are trained for 5 epochs with default initial point, full sample size $N = 10^6$, batch size $B = 1,000$ and the same random seed 123. The boundaries are set as $\{\theta : \|\theta\|_\infty \leq \lambda\}$, where $\lambda \in \{2, 5, 10\}$, since the initial point of the Transformer model satisfies $\|\theta_0\|_\infty = 1$. Because the constraints are always inactive during the whole experiments, SGD and ADAM can be used for unconstrained problems without sacrifice of fairness.

We compare the following 4 algorithms.

- **SGD**: Stochastic Gradient Descent by Robbins and Monro [1951]. We implement this algorithm in Python with learning rate 0.001.
- **ADAM**: Adaptive Moment Estimation method by Kingma and Ba [2015]. We implement this algorithm in Python with learning rate 0.001 and ADAM parameters $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$.
- **GOP**: Greedy Oriented Progression as in Algorithm 2. The tuning parameters are set as $m = 1,000$ and $\delta = 0.05$.
- **ADAM-GOP**: Greedy Oriented Progression with ADAM direction. The parameters are chosen the same as GOP with ADAM parameters $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$.

C Details of Real Data Application

This section provides additional details for the real-data quantile-regression experiment in Section 5. The experiment is designed to compare the optimization behavior of GOP-type methods with standard first-order baselines and the task-specific subGD baseline under the same one-pass data-processing budget.

C.1 Data and Preprocessing

We use the 1980 IPUMS USA sample. After preprocessing, the experiment keeps $N = 3,659,684$ observations and constructs an input vector of dimension $d = 675$. The data are processed in 3,574 chunks. We use the following baseline model:

$$\log(Wage_i) = \beta_0 + \beta_1 Female_i + \beta_2 Educ_i + \beta_3 Female_i \times Educ_i + \theta'_1 X_i + \theta'_2 (X_i \times Female_i) + \varepsilon_i, \quad (32)$$

where $Wage_i$ is a real weekly wage in 1980 terms, $Female_i$ is a female dummy, $Educ_i$ is an education dummy for some college or above, and X_i is a vector of additional control variables. For control variable X_i , we use 12 age group dummies with a four-year interval, 51 states dummies (including D.C.), and their interactions. Note that $(X_i \times Female_i)$ implies that there exist up to 3-way interactions. The model contains 1226 covariates in total.

C.2 Objective Function

For each observation (Y_i, X_i) , we minimize the empirical pinball loss

$$\hat{G}(\theta) = \frac{1}{N} \sum_{i=1}^N \rho_\tau(Y_i - X_i^\top \theta), \quad \rho_\tau(u) = u\{\tau - I(u < 0)\}. \quad (33)$$

The stochastic subgradient used by SGD, subGD, GOP, and Adam-GOP is based on mini-batches or chunks of the data. The optimization region for GOP-type methods is chosen as a hypercube

$$\Gamma = \{\theta \in \mathbb{R}^d : \|\theta\|_\infty \leq \lambda\}, \quad (34)$$

where $\lambda = 10$. The starting index and buffer parameters for GOP-type methods are set to $m = 100$ and $\delta = 0.05$, respectively.

C.3 Algorithms and Hyperparameters

We compare five algorithms: SGD, Adam, subGD, GOP, and Adam-GOP. All methods are initialized at the same starting point and are run for one epoch over the data. The detailed hyperparameter choices are listed below.

- SGD: fixed learning rate $\gamma = 0.01$.
- Adam: fixed learning rate $\gamma = 0.01$, with Adam parameters $(\beta_1, \beta_2, \varepsilon) = (0.9, 0.999, 10^{-8})$.
- subGD: learning rate schedule and averaging rule: $\gamma = \gamma_0 i^{-a}$, $\gamma_0 = 1.0$ and $a = 0.501$.
- GOP: boundary size $\lambda = 10$, starting index $m = 100$, and buffer parameter $\delta = 0.05$.
- Adam-GOP: the same GOP updating rule, but using the Adam direction. The Adam parameters are $(\beta_1, \beta_2, \varepsilon) = (0.9, 0.999, 10^{-8})$.

The batch size or chunk size used by all methods is 1024. The same random seed 123 is used whenever randomness is involved.

C.4 Evaluation Protocol

We report two loss summaries. The first is the mean loss over the whole data pass, which reflects both the speed of convergence and the average objective value encountered along the optimization trajectory. The second is the final smoothed loss, computed from the last smoothing window of the recorded loss history. The smoothing window used in Figure 3 is 50.

The observed whole-pass mean losses are

Method	SGD	Adam	subGD	GOP	Adam-GOP
Mean loss	0.521628	0.254368	0.200096	0.202982	0.197934

(35)

The observed final smoothed losses and running times are

Method	SGD	Adam	subGD	GOP	Adam-GOP
Final smoothed loss	0.198082	0.186610	0.184276	0.182120	0.185031
Time (s)	75.05	79.62	78.36	79.58	77.54

(36)

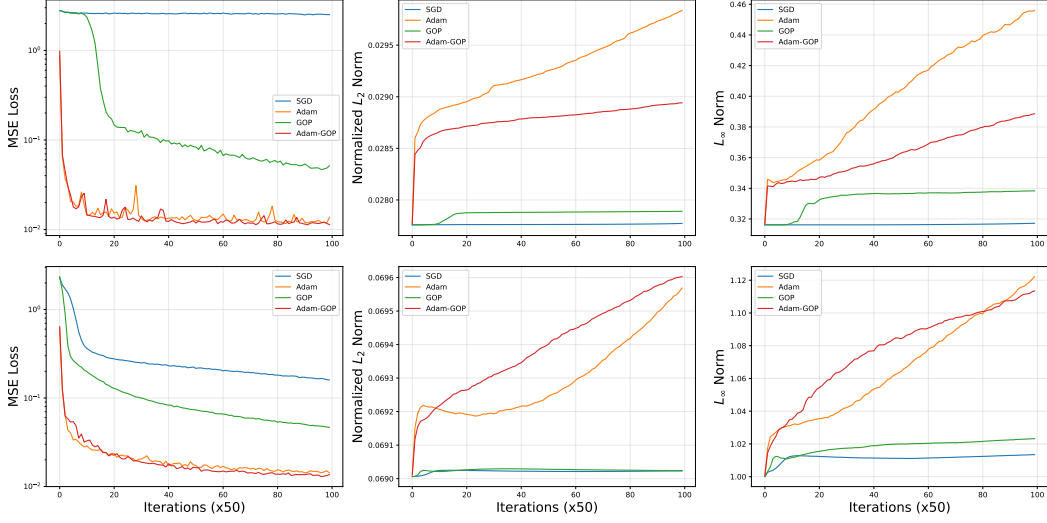


Figure 6: Performance comparison on DeepReLU MLP (first row) and Transformer (second row) in a single run. The first column shows the training loss; the second and third columns track the evolution of parameter norms. GOP variants consistently achieve lower loss. For DeepReLU MLP, the total dimension is $d \sim 2 \times 10^6$ and the training costs are (277, 379, 410, 519) seconds respectively. For Transformer, the total dimension is $d \sim 5 \times 10^5$ and the training costs are (483, 532, 550, 613) seconds respectively.

D Supplemental Numerical Results

In this section, we include supplemental numerical results for the settings described in Section B. Specifically, Figure 4 and Figure 5 show the experimental results for $\lambda = 2$ and $\lambda = 10$ respectively. The models include **DeepReLU MLP**: A 4-layer fully connected network with 64 hidden units (approx. 1.3×10^4 parameters), and **Transformer**: An encoder-only Transformer with 2 layers, 4 heads, and model dimension 32 (approx. 1.7×10^4 parameters). Clearly, in all settings, our proposed algorithms GOP and ADAM-GOP beat their baselines respectively. Based on these results, we empirically show that the performance of our algorithm is robust to the choice of the edge length of hyper-cube Γ .

Next, Figure 6 shows the performance for optimizing higher dimension models. The models include **DeepReLU MLP**: A 8-layer fully connected network with 512 hidden units (approx. 2×10^6 parameters), and **Transformer**: An encoder-only Transformer with 4 layers, 4 heads, and model dimension 128 (approx. 5×10^5 parameters). In this high-dimensional setting, ADAM-GOP slightly beats ADAM in terms of the final loss, while SGD loses to GOP by a large margin. This result shows that even in the case where the dimension is higher than the sample size, our proposed algorithms still win.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [\[Yes\]](#)

Justification: Yes, the main claims of the paper are accurately reflected in both the abstract and introduction. The paper proposes *Greedy Oriented Progression* (GOP), a first-order mini-batch optimization framework for high-dimensional multi-modal stochastic objectives. All claims are theoretically proven and empirically validated throughout the work.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: All the theoretical assumptions about the local convergence behavior of GOP are explicitly mentioned and discussed at length in Sections 3 and A. It also points out the fundamental restrictions on the noise level and the optimization landscape for theoretical validity of the GOP algorithm. These discussions appear in the theoretical analysis sections and remarks following theorems. The detailed discussion of practical limitations are shown in Section B, where the choice of hyperparameters such as the "learning rate" of GOP are explicitly discussed.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: All the assumptions can be found in the theorem statements, and are discussed in main paper, Section 3. All the proofs can be found in appendix Section A.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The complete model specifications along with parameters, are provided in appendix sections B, C and D.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All code to reproduce the results can be found anonymously in github as well as in the supplemental material.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The complete model specifications along with parameters, are provided in appendix sections B, C and D.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The current figures and tables report empirical performance over repeated trials. Since the number of repetitions are large and the differences of empirical performance are significant enough, this omission is not central to the theoretical claims and does not weaken the main conclusions significantly.

The paper does not include hypothesis testing or statistical significance analysis, and error bars are not a focus of the experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The experiments are lightweight and run quickly on a modern laptop. The exact time of execution is reported.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research follows all ethical guidelines. We use publicly available, de-identified IPUMS USA microdata under its terms of use; no new human-subject data were collected.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [\[Yes\]](#)

Justification: The paper mentions certain societal motivations, such as computational efficiency while maintaining statistical validity, in the introduction. Since paper presents work whose goal is to advance the field of Machine Learning, there are many potential societal consequences of our work, none of which we feel must be specifically highlighted in the main text.

Guidelines:

- The answer [\[N/A\]](#) means that there is no societal impact of the work performed.
- If the authors answer [\[N/A\]](#) or [\[No\]](#), they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [\[N/A\]](#)

Justification: The paper does not release any models or datasets with high risk of misuse. All released components are either synthetic or public and pose no privacy or safety risk.

Guidelines:

- The answer [\[N/A\]](#) means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: All baseline methods are standard, and citations to prior work are provided with proper attribution and licensing notices.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The paper introduces a novel stochastic online optimization algorithm, whose performance are both theoretically proven with rigorous proofs and empirically validated with extensive simulations. All the accompanying empirical evidence is documented and released via a GitHub repository in anonymized form. Hyperparameters, dependencies, and usage instructions are all included.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The paper does not involve crowdsourcing or human subject research.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects are involved in this research.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: No large language models (LLMs) were used to produce, analyze, or verify the scientific content of this paper. All methods and results are original and independently verified using rigorous mathematical analysis and custom code.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.